Task 6. Prepare answers to the following questions

1. **What operations do the following functions perform:**

   - **film_in_stock:**

     - returns available film inventory_id list

```
CREATE OR REPLACE FUNCTION public.film in stock(p film id integer, p store id integer,
OUT p film count integer)
 RETURNS SETOF integer    -- function returns film id list
 LANGUAGE sql
AS $function$
     SELECT inventory id
     FROM inventory
     WHERE film id = $1  -- by provided film id
     AND store id = $2   -- and store id
     AND inventory in stock(inventory id);
$function$
;
-- function nests inventory in stock() function which checks inventory availability and
returns boolean value. In WHERE clause 'AND' requires of nested functions value TRUE.
```

   - **film_not_in_stock:**

     - returns not available film inventory_id list

```
CREATE OR REPLACE FUNCTION public.film in stock(p film id integer, p store id integer,
OUT p film count integer)
 RETURNS SETOF integer    -- function returns film id list
 LANGUAGE sql
AS $function$
     SELECT inventory id
     FROM inventory
     WHERE film id = $1  -- by provided film id
     AND store id = $2   -- and store id
     AND NOT inventory in stock(inventory id);
$function$;
-- function nests inventory in stock() function which checks inventory availability and
returns boolean value. In WHERE clause 'AND NOT' requires of nested functions value
FALSE.
```

   - **last_day:**

     - Function returns date according to provided date. Returned date is next month's first day.

```
CREATE OR REPLACE FUNCTION public.last_day(timestamp with time zone)
 RETURNS date -- returns date
 LANGUAGE sql
 IMMUTABLE STRICT
AS $function$
  SELECT CASE
-- this string is for versatility when the date is the last month of the year. If it is a
December, then extracted year + 1
    WHEN EXTRACT(MONTH FROM $1) = 12 THEN
-- explicitly specified operators "operator(pg_catalog.||)" to avoid any potential
conflicts
      (((EXTRACT(YEAR FROM $1) + 1) operator(pg_catalog.||) '-01-01')::date - INTERVAL '1
day')::date
    ELSE
      ((EXTRACT(YEAR FROM $1) operator(pg_catalog.||) '-' operator(pg_catalog.||)
(EXTRACT(MONTH FROM $1) + 1) operator(pg_catalog.||) '-01')::date - INTERVAL '1
day')::date
    END
$function$;
```

- **inventory_in_stock:**

  - by provided inventory_id function checks inentory availability

```
CREATE OR REPLACE FUNCTION public.inventory in stock(p inventory id integer)
RETURNS Boolean --returns boolean type value. TRUE - available, FALSE - not.
LANGUAGE plpgsql
AS $function$
DECLARE
    v rentals INTEGER;
    v out     INTEGER;
BEGIN
    -- the value is assigned to the variable
    SELECT count(*) INTO v rentals
    FROM rental
    WHERE inventory_id = p_inventory_id;
     -- First check. Has inventory ever been rented. If not - function returns TRUE
     (inventory is available) and no further actions is made.
    IF v rentals = 0 THEN
      RETURN TRUE;
    END IF;
    -- the value is assigned to the variable
    SELECT COUNT(rental id) INTO v out
    FROM inventory LEFT JOIN rental USING(inventory id)
    WHERE inventory.inventory id = p inventory id
    AND rental.return_date IS NULL;
    -- If in the first check we got FALSE, then function checks maybe rented inventory is
    returned. If it's not returned - function RETURNS FALSE.
    IF v out > 0 THEN
      RETURN FALSE;
    ELSE
      RETURN TRUE;
    END IF;
END $function$
;
```

- **inventory_held_by_customer:**

  - by provided inventory ID, function returns customer's ID who hasn't returned inventory yet.

```
CREATE OR REPLACE FUNCTION public.inventory_held_by_customer(p_inventory_id integer)
 RETURNS integer -- returns integer type
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_customer_id INTEGER;
BEGIN
  -- the customer_id value is assigned to the variable
  SELECT customer_id INTO v_customer_id
  FROM rental
   -- if inventory is not returned yet
  WHERE return_date IS NULL
  AND inventory_id = p_inventory_id;
-- returns inventory ID
  RETURN v_customer_id;
END $function$
;
```

- **get_customer_balance:**
  - by provided customers ID and date, function evaluates customers rentfees, overfees, payments and returns customers numeric balance (rentfees+overfees-payments)

```
CREATE OR REPLACE FUNCTION public.get_customer_balance(p_customer_id integer,
p_effective_date timestamp with time zone)
 RETURNS numeric -- returns customers balance
 LANGUAGE plpgsql
AS $function$
DECLARE
    v_rentfees DECIMAL(5,2); --#FEES PAID TO RENT THE VIDEOS INITIALLY
    v_overfees INTEGER;      --#LATE FEES FOR PRIOR RENTALS
    v_payments DECIMAL(5,2); --#SUM OF PAYMENTS MADE PREVIOUSLY
BEGIN
   -- the rentfees value is assigned to the variable
    SELECT COALESCE(SUM(film.rental_rate),0) INTO v_rentfees
    FROM film, inventory, rental
    WHERE film.film_id = inventory.film_id
      AND inventory.inventory_id = rental.inventory_id
      AND rental.rental_date <= p_effective_date
      AND rental.customer_id = p_customer_id;
   -- the overfees value is assigned to the variable
    SELECT COALESCE(SUM(CASE
                        -- one daollar for every overdue day
                            WHEN (rental.return_date - rental.rental_date) >
                                 (film.rental_duration * '1 day'::interval)
                            THEN EXTRACT(epoch FROM ((rental.return_date -
                                 rental.rental_date) - (film.rental_duration *
                                 '1 day'::interval)))::INTEGER / 86400 -- *
                                 1 dollar
                            ELSE 0
                         END),0)
    INTO v_overfees
    FROM rental, inventory, film
    WHERE film.film_id = inventory.film_id
      AND inventory.inventory_id = rental.inventory_id
      AND rental.rental_date <= p_effective_date
      AND rental.customer_id = p_customer_id;
   -- the payments value is assigned to the variable
    SELECT COALESCE(SUM(payment.amount),0) INTO v_payments
    FROM payment
    WHERE payment.payment_date <= p_effective_date
    AND payment.customer_id = p_customer_id;
    -- calculation is made and result is returned
    RETURN v_rentfees + v_overfees - v_payments;
END
$function$
;
```

- **rewards_report:**
  - **what is tmpSQL variable for in 'rewards_report' function?**
    - by provided min_monthly_purchases and min_dollar_amount_purchases benchmarks, function returns all customer table rows by customer ID's who matches requirements for reword. Data is evaluated from current date till past 3 months.
    - tmpSQL variable is assigned for insert query of filtered customer ID's.
    - insertion by tmpSQL variable is executed in to temporal table "tmpCustomer" which is created in function before.
    - "tmpCustomer" table data is used for filtering "customer" table's data by "tmpCustomer" inner join with "customer" table on customer_id.
    - At the end "tmpCustomer" is dropped.

```
CREATE OR REPLACE FUNCTION public.rewards_report(min_monthly_purchases integer, min_dollar_amount_purchased
numeric)
 RETURNS SETOF customer --mean that function returns part of customer table's rows
 LANGUAGE plpgsql
 SECURITY DEFINER
AS $function$
DECLARE
    last_month_start DATE; -- declaration of start and end dates for calculation
    last_month_end DATE;
    rr RECORD;
    tmpSQL TEXT; -- declaration of tmpSQL variable
BEGIN

Comment have been in code: Some sanity checks.
    IF min_monthly_purchases = 0 THEN
        RAISE EXCEPTION 'Minimum monthly purchases parameter must be > 0';
    END IF;
    IF min_dollar_amount_purchased = 0.00 THEN
        RAISE EXCEPTION 'Minimum monthly dollar amount purchased parameter must be > $0.00';
    END IF;

    last_month_start := CURRENT_DATE - '3 month'::interval;
    last_month_start := to_date((extract(YEAR FROM last_month_start) || '-' || extract(MONTH
                        FROM last_month_start) || '-01'),'YYYY-MM-DD');
    last_month_end := LAST_DAY(last_month_start);

Comment have been in code: Create a temporary storage area for Customer IDs.
    CREATE TEMPORARY TABLE tmpCustomer (customer id INTEGER NOT NULL PRIMARY KEY);
Comment have been in code: Find all customers meeting the monthly purchase requirements
    -- insert query of filtered customer IDs by provided criteria of 3 months from current date
    tmpSQL := 'INSERT INTO tmpCustomer (customer_id) -- tmpSQL variable is assigned for insert query
        SELECT p.customer_id
        FROM payment AS p
        WHERE DATE(p.payment_date) BETWEEN '||quote_literal(last_month_start) ||' AND '||
quote_literal(last_month_end) || '
        GROUP BY customer_id
        HAVING SUM(p.amount) > '|| min_dollar_amount_purchased || '
        AND COUNT(customer_id) > ' ||min_monthly_purchases ;

    EXECUTE tmpSQL; -- tmpSQL variable = INSERT is executed
  --Iteration of filtered customers data by "tmpCustomer" table's ID's
      FOR rr IN EXECUTE 'SELECT c.* FROM tmpCustomer AS t INNER JOIN customer AS c ON t.customer_id =
c.customer_id' LOOP
        RETURN NEXT rr; --returning set of customer table
    END LOOP;

Comment have been in code: Clean up
    tmpSQL := 'DROP TABLE tmpCustomer'; --dropped temporal table
    EXECUTE tmpSQL;
RETURN;
END
$function$
;
```

- **can this function be recreated without EXECUTE statement and dynamic SQL? Why?**
  - Function can be made without EXECUTE statement because 'dynamic' query `'SELECT c.* FROM tmpCustomer AS t INNER JOIN customer AS c ON t.customer_id = c.customer_id'` is static by itself and can be written just as simple query.
  - Temporal table is redundant element, so we don't need tmpSQL variable and it's execution too. We can make tmpSQL as filter subquery.

Example without EXECUTE:

```
CREATE OR REPLACE FUNCTION public.rewards_report(min_monthly_purchases integer, min_dollar_amount_purchased
NUMERIC, choose_date_from DATE)
 RETURNS SETOF customer
 LANGUAGE plpgsql
 SECURITY DEFINER
AS $function$
DECLARE

BEGIN
   --Comment have been in code: Some sanity checks.
    IF min_monthly_purchases = 0 THEN
        RAISE EXCEPTION 'Minimum monthly purchases parameter must be > 0';
    END IF;
    IF min_dollar_amount_purchased = 0.00 THEN
        RAISE EXCEPTION 'Minimum monthly dollar amount purchased parameter must be > $0.00';
    END IF;

   RETURN QUERY
   SELECT *
   FROM customer
   WHERE customer_id IN
        (
        SELECT p.customer_id
        FROM payment AS p
        WHERE DATE(p.payment_date) < choose_date_from
        AND DATE(p.payment_date) > choose_date_from - '3 month'::INTERVAL
        GROUP BY customer_id
        HAVING SUM(p.amount) > min dollar amount purchased
        AND COUNT(customer_id) > min_monthly_purchases
        );
END;
$function$;

--FUNCTION CHECK
SELECT * FROM public.rewards_report(min_monthly_purchases=>1, min_dollar_amount_purchased=>1,
choose_date_from=>'2017-06-01')
```

2. What does 'last_updated' function do? Where is it used?

- 'last_updated' function is used in every schema's table trigger eg.:

```
CREATE TRIGGER last_updated BEFORE
UPDATE
    ON
    public.actor FOR EACH ROW EXECUTE FUNCTION last_updated();
```

- 'last_updated' function returns last update (`CURRENT_TIMESTAMP`) date for each row.

```
CREATE OR REPLACE FUNCTION public.last_updated()
 RETURNS trigger --returns triger
 LANGUAGE plpgsql
AS $function$
BEGIN
    -- NEW is a pseudo-record name that refers to the new table row for insert and update
operations in row-level triggers. Its usage is :NEW.column, where column is the name of a
column in the table on which the trigger is defined.
    NEW.last_update = CURRENT_TIMESTAMP; -- updated value type
    RETURN NEW;
END $function$
;
```

3. **Why does 'rewards_report' function return 0 rows?**
Correct and recreate the function, so that it's able to return rows properly.

Because of:

- Variable `last month start` is used to filter data;
  - `last month start := CURRENT_DATE - '3 month'::interval;`
- Payment_date data in payment table is from *2017-01-24 20:21:56.996 +0200* to *2017-06-01 00:00:00.000 +0300;*
- *So Current_date – 3 months = 2023-11-09 – 3 months = 2023-08-09.*
- *By this implemented logic we can't reach existing data.*

Solution:

- Added additional function's property "`IN date_from DATE`"
- Updated "`last month_start`" assignment to "`last_month_start := date_from - '3 month'::interval;`"

```
CREATE OR REPLACE FUNCTION public.rewards_report(min_monthly_purchases integer,
min_dollar_amount_purchased NUMERIC, IN date_from DATE)
 RETURNS SETOF customer
 LANGUAGE plpgsql
 SECURITY DEFINER
AS $function$
DECLARE
    last_month_start DATE;
    last_month_end DATE;
rr RECORD;
tmpSQL TEXT;
BEGIN

    /* Some sanity checks... */
    IF min_monthly_purchases = 0 THEN
        RAISE EXCEPTION 'Minimum monthly purchases parameter must be > 0';
    END IF;
    IF min_dollar_amount_purchased = 0.00 THEN
        RAISE EXCEPTION 'Minimum monthly dollar amount purchased parameter must be > $0.00';
    END IF;

    last_month_start := date_from - '3 month'::interval;
    last_month_start := to_date((extract(YEAR FROM last_month_start) || '-' || extract(MONTH
FROM last_month_start) || '-01'),'YYYY-MM-DD');
    last_month_end := LAST_DAY(last_month_start);
/*
    Create a temporary storage area for Customer IDs.
    */
    CREATE TEMPORARY TABLE tmpCustomer (customer_id INTEGER NOT NULL PRIMARY KEY);

    /*
    Find all customers meeting the monthly purchase requirements
    */

    tmpSQL := 'INSERT INTO tmpCustomer (customer_id)
        SELECT p.customer_id
        FROM payment AS p
        WHERE DATE(p.payment_date) BETWEEN '||quote_literal(last_month_start) ||' AND '||
quote_literal(last_month_end) || '
        GROUP BY customer_id
        HAVING SUM(p.amount) > '|| min_dollar_amount_purchased || '
        AND COUNT(customer_id) > ' ||min_monthly_purchases ;

    EXECUTE tmpSQL;
```

<.....>

4. **Is there any function that can potentially be removed from the dvd_rental codebase? If so, which one and why?**

   Last_day function, because it's not used at all and doesn't provide any useful info for dvdrental matter at the moment.

5. **The 'get_customer_balance' function describes the business requirements for calculating the client balance. Unfortunately, not all of them are implemented in this function. Try to change function using the requirements from the comments.**

   - Fulfilled condition: `IF A FILM IS MORE THAN RENTAL_DURATION * 2 OVERDUE, CHARGE THE REPLACEMENT_COST`
   - Added lines highlighted in red.

<……>

```
SELECT COALESCE(SUM(CASE
                WHEN (rental.return_date - rental.rental_date) > (film.rental_duration * '2 day'::interval)
                THEN film.replacement_cost
                WHEN (rental.return_date - rental.rental_date) > (film.rental_duration * '1 day'::interval)
                THEN EXTRACT(epoch FROM ((rental.return_date - rental.rental_date) - (film.rental_duration * '1 day'::interval)))::INTEGER / 86400 -- * 1 dollar
                ELSE 0
               END),0)
```

<……>

6. **How do 'group_concat' and '_group_concat' functions work? (database creation script might help) Where are they used?**

   - 'group_concat' were used for actor name and surname concatenation in:
     - public.actor_info view
     - public.film_list view
     - public.nicer_but_slower_film_list view

   - '_group_concat' function returns text. If there are provided 2 texts – they are concatenated.
   - '_group_concat' can be used to concatenate values of same row

```
CREATE OR REPLACE FUNCTION public._group_concat(text, text)
 RETURNS text
 LANGUAGE sql
 IMMUTABLE
AS $function$
SELECT CASE
  WHEN $2 IS NULL THEN $1 --when second property is NULL then returns first properties text
  WHEN $1 IS NULL THEN $2 --when first property is NULL then returns second properties text
  ELSE $1 || ', ' || $2 --if both properties is provided they are concatenated
END
$function$
;
```

   - 'group_concat' - aggregate function is useful when you want to concatenate values within groups of rows. Column values will be provided as text separated by ', ' and grouped by as ordered in query.
   - 'group_concat' cant work without '_group_concat'

```
CREATE OR REPLACE AGGREGATE public.group_concat(pg_catalog.text -- data type specified by PostgreSQL database)
(
   SFUNC = _group_concat, --transition state function - It is executed in an aggregation as many times as there are rows to aggregate.
   STYPE = text --intermediate state's data type
);
```